

Paper2Repro: Execution-Grounded Paper to Code Reproduction

Anonymous ACL submission

Abstract

Automatic generation of executable code from scientific papers is crucial for both research aim and the practical adoption of new methods in industry. Existing paper-to-code agents can produce repositories that appear faithful and may score highly under LLM-as-a-judge evaluation, but such apparent quality does not ensure runnability or scientific reproducibility. We propose a rubric- and execution-guided paper-to-code framework that verifies generated repositories through intermediate-value tests, runtime checks, and benchmark-level reproduction criteria. Our workflow parses papers into structured JSON, generates initial repositories through a planning-analysis-coding pipeline, constructs paper-specific rubrics and `pytest` tests, extracts runtime requirements, and packages the resulting task into a Harbor-style evaluation environment. On 22 PaperBench papers, our prototype substantially improves runnability and reveals a large gap between executable repositories and scientifically reproducible repositories. These results suggest that paper-to-code evaluation should move beyond LLM-as-a-judge scores toward execution-grounded verification of algorithmic and experimental fidelity.

1 Introduction

Reproducibility is essential for scientific progress, allowing researchers to verify findings, build on previous work, and push the boundaries of knowledge (Open Science Collaboration, 2015; Baker, 2016; Pineau et al., 2021). In machine learning, however, reproducing experimental results from a paper alone remains a persistent challenge: documentation is often incomplete, hyperparameters are under-specified, and a substantial fraction of accepted papers release no accompanying code (Seo et al., 2025). This forces researchers to invest considerable effort reverse-engineering methods from prose, slowing the pace of scientific dissemination.

Recent Large Language Models (LLMs) have shown strong capabilities in both understanding scientific text and generating code (Dubey et al., 2024; OpenAI, 2024; Reid et al., 2024), prompting a wave of paper-to-code agents that aim to translate a research paper directly into an executable repository (Seo et al., 2025; Starace et al., 2025; Siegel et al., 2024a). Systems such as PaperCoder (Seo et al., 2025) decompose this task into planning, analysis, and coding stages, and emit repositories that are structurally complete and stylistically convincing. Under LLM-as-a-judge evaluation, such repositories routinely receive high faithfulness scores. The implicit conclusion is that the generated code is a reasonable proxy for the paper’s implementation.

We argue that this conclusion conflates three increasingly strict properties that should be kept distinct: whether a repository *looks* faithful to the paper, whether it actually *runs*, and whether it *reproduces* the paper’s experimental behavior. A simple analogy is egg fried rice. A finished plate may look right, but a positive visual judgment does not entail that the cook understood the recipe, followed the correct procedure, or produced a dish with the expected taste. A judge that only inspects the plate cannot detect a missing ingredient, an inverted step, or a heat profile that differs from the original. The same holds for paper-to-code systems: a repository may use the wrong loss term, omit a key normalization, depend on an uninstalled package, or execute successfully yet fail to match the reported baseline. Existing benchmarks largely measure surface-level repository quality and do not separate these three properties, leaving open the question of whether current paper-to-code agents are reproducing science or merely producing code that resembles it.

To address this gap, we propose that paper-to-code evaluation should be *execution-grounded* rather than *appearance-based*, and we present a prototype framework that realizes this principle. Given a paper, the framework runs three parallel

agents: a **coding agent** that emits an initial repository; a **rubric and unit-test generation agent** that converts paper claims into `pytest` cases targeting intermediate quantities and final outputs; and a **runtime-information extraction agent** that records hardware, package, dataset, and hyperparameter requirements. A subsequent code-running and amelioration stage uses rubric checks, test outcomes, and execution traces to revise the repository. The final artifact is packaged as a TerminalBench-style (Stanford and Anthropic, 2025) reproducibility task that can be replayed by external agents. The novelty here is not the multi-agent decomposition itself, but the shift in what counts as evidence of reproduction: rather than asking only whether the final result *looks* right, the framework verifies that the repository computes the correct intermediate quantities, runs to completion, and satisfies benchmark-level criteria.

We organize our investigation around three research questions:

RQ1 Does LLM-as-a-judge repository quality reflect actual runnability and scientific reproducibility?

RQ2 Can paper-specific rubrics and intermediate-value tests better identify algorithmic implementation errors than end-to-end output checks?

RQ3 Can static and runtime feedback meaningfully improve generated paper-to-code repositories?

We make three contributions. *First*, we identify and characterize a gap between apparent repository quality, executability, and scientific reproducibility in current paper-to-code agents. *Second*, we propose a rubric- and execution-guided workflow that decomposes paper-to-code reproduction into paper understanding, code generation, rubric and unit-test generation, runtime-information extraction, static amelioration, and TerminalBench-style task packaging. *Third*, we evaluate the prototype on 22 papers from PaperBench (Starace et al., 2025) and show that making generated repositories runnable is substantially easier than reproducing paper-level experimental claims, underscoring the need for intermediate-value verification and stronger runtime-guided refinement.

2 Related Work

Paper-to-code generation. PaperCoder (Seo et al., 2025) is the closest baseline to our work. It

decomposes paper-to-code into planning, analysis, and coding stages and shows that structured repository generation improves implementation quality over single-prompt baselines. However, PaperCoder’s main evaluation centers on repository-level faithfulness as judged by an LLM, with comparatively limited analysis of executability or experimental reproduction. Our framework adopts the planning-analysis-coding skeleton but extends it with paper-specific rubric generation, unit-test generation, runtime-information extraction, and a code-running amelioration loop intended to verify the generated repository against execution-grounded criteria.

Reproduction benchmarks. A growing body of work benchmarks paper-to-code systems and code-using agents. PaperBench (Starace et al., 2025) measures whether agents can reproduce machine-learning papers from scratch using hierarchical rubrics, and provides the papers we use in our evaluation. SciReplicate-Bench (Xiang et al., 2025) focuses on reproducing algorithmic procedures from method sections. CORE-Bench (Siegel et al., 2024b) evaluates the reproducibility of computational research more broadly, while REPRO-Bench (Hu et al., 2025) targets end-to-end reproduction with explicit execution checks. Compared to these benchmarks, our focus is not on building a static benchmark but on a *generative* workflow that turns a paper into a reproducible task on the fly, while inheriting their key insight that execution must be part of evaluation.

Agentic terminal benchmarks. Terminal-Bench (Stanford and Anthropic, 2025) evaluates agents on realistic terminal tasks using containerized environments, natural-language instructions, test suites, oracle solutions, and time limits. Our Harbor-style packaging adapts this evaluation philosophy to paper-to-code reproduction: the final artifact is not a static repository but a self-contained task that bundles paper, code, runtime specification, and tests.

Test-time refinement. Iterative self-debugging (Chen et al., 2023; Tian et al., 2024) and best-of- N sampling approaches (Brown et al., 2024) have shown that test-time feedback can improve LLM code generation. Our amelioration loop is in this lineage, but uses *paper-specific* rubrics and intermediate-value tests as the feedback signal, rather than generic correctness oracles or arbitrary

unit tests.

3 Method

In this section, we formalize the task of paper-to-code reproduction with paper-grounded evaluation, and then propose our multi-agent framework, which extends repository-level code generation with rubric-driven unit testing, runtime-aware execution, and benchmark packaging.

The goal of our task is to automatically produce a code repository that not only implements the methods and experiments described in a scientific paper, but also satisfies a paper-derived evaluation standard that distinguishes superficially plausible code from code that faithfully reproduces the paper’s claims (Pineau et al., 2021; Starace et al., 2025). Formally, given a paper R , we define a mapping $\mathcal{M}$ that yields four artifacts:

$$\mathcal{M}(R) = (C, U, I, B),$$

where $C = \{c_1, \dots, c_n\}$ is the generated code repository, U is the paper-specific rubric together with an executable unit-test suite, I is the structured runtime information required to execute the repository (including dataset and environment specifications), and B is a Harbor-style reproducibility task bundle. The most straightforward approach to instantiating $\mathcal{M}$ is to instruct an LLM to generate C alone, as in prior repository-level code generation work (Seo et al., 2025); yet, C alone underspecifies reproducibility, because it conflates the question of whether the code runs with the question of whether the code computes the right quantities. Therefore, we decompose $\mathcal{M}$ into five coordinated stages, with U and I derived directly from R rather than from C , so that the paper—not the generated code—remains the source of ground truth. Formally, the overall pipeline is:

Code Generation: $(C, S) = \mathcal{M}_{\text{code}}(R)$,

Rubric/Tests: $U = \mathcal{M}_{\text{rub}}(R, S)$,

Runtime/Dataset Info: $I = \mathcal{M}_{\text{run}}(R, S)$,

Amelioration: $C^* = \mathcal{M}_{\text{amel}}(C, U, I)$,

Packaging: $B = \mathcal{M}_{\text{pkg}}(R, C^*, U, I)$,

where S denotes a set of structured intermediate artifacts produced during code generation and reused by downstream agents. The overall pipeline is shown in Figure 1. We then describe how each stage is instantiated below.

3.1 Code Generation Phase

The code generation phase is largely the same as PaperCoder (Seo et al., 2025), which we adopt with two modifications. First, we prepend a pre-processing step that converts the input PDF into a cleaned JSON representation R' (`paper.json`), with citation spans, bibliography entries, headers, and footers removed, so that all downstream agents reason over the same canonical input. Second, in addition to the repository C , this phase explicitly exposes a set of structured intermediate artifacts

$$S = (R', P, A, E),$$

where P is the planning trajectory, A is the per-file analyses, and E is a paper-grounded extraction containing algorithmic specifications, equations, datasets, baselines, and reported metrics. These artifacts are reused downstream by the rubric agent (for paper-grounded test design), the unit-test agent (for alignment with the planned implementation), and the runtime/dataset agent (for environment and dataset specifications). We note that S is derived from R' , not from C ; conditioning downstream agents on S therefore does not implicitly relax the evaluation standard to whatever the coding agent happened to produce.

Formally, given a paper R , we define

$$\begin{aligned} \mathcal{M}_{\text{code}}(R) := & \mathcal{M}_{\text{debug}} \circ \mathcal{M}_{\text{depcode}} \circ \mathcal{M}_{\text{ana}} \\ & \circ \mathcal{M}_{\text{plan}} \circ \mathcal{M}_{\text{pre}}(R) \rightarrow (C, S). \end{aligned}$$

where the four core substeps follow PaperCoder closely.

Planning. The planning agent extracts an overall plan, an architecture design, a logic design, and an initial `config.yaml`, yielding $P = \text{LLM}(\mathcal{T}_{\text{plan}}(R'))$.

Per-file analysis. For each file f_i in the planned repository, the analysis agent writes a natural-language blueprint a_i before any code is emitted, yielding $A = \{a_i\}_{i=1}^{|F|}$.

Dependency-aware coding. Files are generated in dependency order, with $c_i = \text{LLM}(\mathcal{T}_{\text{code}}(R', P, f_i, a_i, \{c_1, \dots, c_{i-1}\}))$, yielding $C = \{c_i\}_{i=1}^{|F|}$.

Debugging. If execution fails during amelioration, the debugging agent proposes minimal search-replace patches rather than rewriting whole files, preserving repository structure and reducing regressions.

We refer the reader to Seo et al. (2025) for full details of the four core substeps. Our contribution

in this phase is the additional extraction of E and the explicit downstream interface through S , which together enable the paper-grounded rubric, unit-test, and dataset agents described next.

3.2 Rubric and Unit-Test Generation

While the code generation phase answers *what* to build, the rubric stage specifies *what successful reproduction means*, and crucially does so without observing the generated repository. Conditioned on the paper R and the structured artifacts S – but not on C – the rubric agent produces a structured rubric and an executable `pytest` suite:

$$\mathcal{M}_{\text{rub}}(R, S) := \text{LLM}(\mathcal{T}_{\text{rub}}(R, S)) \rightarrow U = (Q, V),$$

where Q is a rubric broken down into code-development, execution, and result-analysis items, and V is the unit-test suite that operationalizes Q . Conditioning on (R, S) rather than C is deliberate: if the rubric were conditioned on C , the evaluation standard could be implicitly relaxed to fit whatever the coding agent produced, and the paper would no longer serve as ground truth. The structured artifacts in S —in particular the equations, algorithmic specifications, and reported metrics in E —let the rubric agent ground individual tests in specific equations and quantities of the paper rather than in vague natural-language descriptions. The generated tests fall into two levels.

Intermediate-value tests. These tests check algorithmic internals—loss terms, diffusion coefficients, normalization invariants, tensor shapes, or values derived from specific equations in the paper. They are the diagnostically richest part of the framework: a system can have the right final number for the wrong reason, but it is much harder to silently match a battery of intermediate quantities.

Final-comparison tests. These tests run benchmark-level checks: does the generated method match or outperform a stated baseline on the dataset and metric described in the paper? Final-comparison tests answer the externally visible question, while intermediate-value tests answer the internally faithful question.

We note that this division is the core methodological contribution of our framework: rather than asking “does the final result look right?”, it asks “did the implementation compute the important internal quantities correctly, *and* did it also produce the right final result?”.

3.3 Runtime and Dataset Information Extraction

Runtime setup is a different problem from code generation, and we treat it as such. Knowing that a paper requires CUDA 12 and a specific Torch version is not the same as knowing how to implement the model, and conflating the two leads to repositories that look correct in isolation but cannot be reproduced. We therefore introduce a dedicated agent that, conditioned on the paper R and the structured artifacts S , extracts execution and dataset requirements into a structured artifact I :

$$\mathcal{M}_{\text{run}}(R, S) := \text{LLM}(\mathcal{T}_{\text{run}}(R, S)) \rightarrow I,$$

where I contains hardware requirements (e.g., GPU memory, CUDA version), package and Docker requirements, datasets and benchmarks (with download endpoints and split specifications), training requirements (e.g., epochs, batch size), hyperparameters and random seeds, and the metrics and reported results to be matched. Reusing the dataset and metric fields already extracted in E avoids re-parsing the paper from scratch, and separating runtime information into its own structured artifact makes it possible to construct, verify, and reuse the environment independently of the code.

3.4 Code Running and Amelioration

The amelioration stage is intended to improve the repository using the rubric items in U , the runtime information in I , and execution feedback.

Static amelioration loop. The static loop takes the generated repository together with the paper-specific rubric and checks code-development items that can be judged without execution—for instance, that the loss function contains a required penalty term, that the configuration file uses the correct hyperparameter, or that a named module exists at the expected location. When a rubric item fails, the LLM proposes minimal search–replace patches and the loop iterates. Many reproduction errors are visible in source code and can be corrected cheaply at this stage, before any training run is launched.

Runtime run-and-fix loop The full runtime loop should: (1) read I ; (2) build or configure the environment; (3) download datasets and baselines; (4) run `reproduce.sh`; (5) execute the generated `pytest` suite V ; (6) read `scores/scores.json`; (7) feed failing tests and logs back to the debugger $\mathcal{M}_{\text{debug}}$; and (8) patch and rerun until tests pass or the iteration cap

is reached. The current codebase contains most components in isolation: dataset downloading, debugging via search-replace patches, unit-test execution, and Harbor packaging.

4 Experimental Setup

We now describe the experimental setup and the metrics used to distinguish appearance-based from execution-grounded reproduction quality.

Papers. We evaluate the prototype on 22 papers drawn from PaperBench (Starace et al., 2025), spanning a mix of deep-learning, optimization, and benchmark-style contributions. The papers were selected to cover a range of structural complexity and dataset requirements.

Backbone. The backbone model for all agents is GPT-5.2. Calls are routed through the same client and use a shared system-prompt template per stage; only the stage-specific instructions $\mathcal{T}_{\text{stage}}$ and the cleaned input R' differ across calls.

Environment. Each generated repository is packaged into a Harbor-style task with paper JSON, generated code, runtime information, `pytest` tests, and a weighted reward script. Tasks are run under a 30-minute timeout on a local machine with an RTX 4060.

Metrics. We report three metrics that together separate superficial faithfulness from execution-grounded reproduction: (i) *LLM Judge Score*, evaluated by GPT-5.2 against a set of generated rubrics; (ii) *Runnable Rate*, whether the repository can execute without fatal errors; and (iii) *Reproduction Score*, a weighted score over code-development, execution, and result-analysis tests; we do not separately report an intermediate-value pass rate, since it is subsumed by this score.

5 Results

We report preliminary results on a 22-paper subset of PaperBench together with BE-CBO, used as an in-depth case study. We treat these numbers as a sanity check on the framework rather than as a final evaluation; absolute values are likely to shift once the runtime run-and-fix loop is completed and dataset and baseline retrieval mature.

5.1 Generated Repo Details

To probe how closely generated repositories obey the paper that produced them, we compared three Paper2Repro outputs against their corresponding papers, rubrics, and official reference repositories:

BaM, APT (Adaptive Pruning and Tuning), and BBOX-ADAPTER. The reference repositories are used as a behavioral calibration target, not as a strict file-template oracle. Scores are rubric-weighted implementation-faithfulness estimates, so they reflect both code coverage and evidence that the paper’s experiments were actually reproduced. Table 1 gives the headline numbers, and the paragraphs below discuss each repository in turn.

BaM: Batch and Match. The BaM repository is the strongest of the three in terms of core mathematical logic. It implements BaM, ADVI, Score-VI, Fisher-VI, and GSM-style algorithms with full-covariance Gaussian variational families, synthetic target distributions, per-iteration metrics, and shared experiment infrastructure. This matches the paper’s central claim that BaM is a score-divergence-based black-box variational inference method with closed-form Gaussian-family updates. Most paper-obedience gaps lie in experimental detail rather than the algorithm skeleton: there is no committed evidence of the required long seeded runs, grid-search sweeps, final figures, or reproduced quantitative results. PosteriorDB support is mostly generic scaffolding—target-specific log-posterior and score functions for the paper’s models are absent or replaced by toy mechanisms—and the deep generative experiment is partial, with CIFAR loading, VAE-style code, posterior scoring, and reconstruction metrics in place but no trained decoder artifacts or completed reconstruction outputs. BaM therefore obeys the paper at the level of method structure, but not at the level of full experimental reproduction.

APT: Adaptive Pruning and Tuning. The APT repository implements a plausible adaptive-pruning system. It contains an APT-like adapter layer with frozen base weights, input/output masks, dynamic rank expansion, salience-density pruning, rank allocation, two-stage training, distillation scaffolding, evaluation utilities, and RoBERTa/T5 configurations that encode many paper hyperparameters, making the repository recognizable as an implementation of the paper’s high-level idea: adaptively tune salient parameters while pruning unimportant structure to improve training and inference efficiency. The main weaknesses are paper-critical details: baselines are local proxies rather than the specified Mask Tuning and CoFiPruning implementations, and several method details diverge from the paper or reference behavior, including effec-

Paper	Score	Leaves	Implementation profile	Paper faithfulness
BaM	0.308	789	Strongest on core variational-inference algorithms; weak on executed experiments and real PosteriorDB/deep-generative reproduction.	Partially faithful: the main algorithmic logic is present, but paper-scale evidence is mostly missing.
APT	0.246	123	Coherent adaptive-pruning framework with masks, salience scoring, dynamic ranks, and training utilities; empirical evidence is minimal.	Weakly faithful: captures the broad APT idea, but diverges on several paper-critical details and lacks reported paper results.
BBOX-ADAPTER	0.290	279	Modular black-box adapter pipeline with pair scoring, online stores, NCE-style training, reranking, and beam search; configs and results are incomplete.	Partially faithful: architecture is recognizable, but training/evaluation details and reproduced tables are largely absent.

Table 1: Rubric-weighted summary of three generated repositories. “Score” is the weighted implementation-faithfulness estimate over the rubric leaves produced for that paper; “Leaves” is the number of leaf rubric items used to compute it.

tive adapter scaling, multi-head attention mask indexing, outlier salience, Equation-9-style adapter salience, layer mapping for distillation, and KL prediction distillation. The empirical side is especially weak: the only observed run is a toy tiny-random RoBERTa SST-2 run with no pruning, while the paper requires 60% sparsity experiments, relative accuracy analysis, efficiency claims, and ablations. APT therefore captures the broad engineering direction of the paper, but does not obey it closely on detailed mechanics or experimental evidence.

BBOX-ADAPTER. The BBOX-ADAPTER repository contains a substantial adaptation pipeline: scalar question-answer pair scoring, ranking/NCE-style adapter training, online positive/negative example storage, single-step reranking, sentence-level full-step beam search, dataset preparation, and metric utilities. It also recognizes several important paper settings from Appendix H.2, including maximum generation length 512, temperature 1.0, learning rate 5×10^{-6} , batch size 64, 6000 training steps, AdamW, and beam size 3. Dataset manifests show materialized GSM8K, StrategyQA, TruthfulQA, and text-only ScienceQA splits. The repository nonetheless falls short of paper-faithful reproduction: there are no committed adapter checkpoints, training logs, evaluation summaries, or reproduced numbers for the paper’s main tables; the default configuration is not runnable as-is because config keys expected by the loader do not match the provided YAML; the ranking loss path has an indentation bug affecting `RankingNCELoss.forward`; and the Equation 3 score regularizer is effectively disabled by default because the α value is not

resolved from the expected configuration location. Several paper components—MLM ablation, fine-tuning baselines, plug-and-play results, cost trends, and the Mixtral white-box extension—are stubs or planned aggregations rather than completed experiments.

Cross-paper interpretation. Across all three repositories, the dominant pattern is the same: generated code often captures the high-level method and implements plausible scaffolding, but falls short on paper-specific detail and experimental proof. BaM is most convincing on core algorithmic logic; APT is most coherent as an engineering framework for pruning and tuning, yet shows the largest gap between method scaffold and paper-scale results; BBOX-ADAPTER has the broadest pipeline for a black-box LLM adaptation workflow, but suffers from configuration and loss-path issues that make reproducibility fragile. These comparisons argue that generated repositories should be treated as starting points for reproduction rather than as faithful reproductions of the papers themselves: useful for studying how much of a method an automated coding system can reconstruct, but requiring substantial human correction before they can support paper-level empirical claims.

5.2 Paper2Code vs. Paper2Repro

Table 2 compares Paper2Code (our re-implementation of the planning-analysis-coding baseline) with Paper2Repro (our framework) on four metrics: runnable rate, LLM judge score on the coding stage, intermediate-value pass rate, and baseline-outperformance rate.

The most striking row is runnability. Under our

Metric	P2C	P2R
Runnable Rate	0%	100%
LLM Judge (coding)	65%	85%
Intermediate-Value Score	—	72%
Baseline Outperformance	—	15%

Table 2: Paper2Code (P2C) vs. Paper2Repro (P2R) on 22 PaperBench papers. Intermediate-value and baseline metrics are not reported for P2C because none of its repositories execute end-to-end under our protocol.

evaluation protocol, none of the 22 P2C repositories run end-to-end within the 30-minute timeout, while all 22 P2R repositories do. Two factors drive this gap: the runtime-information agent fixes most environment-level mismatches before execution, and the static amelioration loop catches the structural bugs—missing imports, mis-spelled module paths, mis-typed configuration keys—that otherwise prevent a repository from starting. Because no P2C repository runs, intermediate-value and baseline metrics cannot be evaluated for P2C, and the corresponding cells are left empty.

The second row shows that P2R also improves the LLM judge score on the coding stage from 65% to 85%. This is consistent with the original Paper2Code finding that decomposed generation produces faithful-looking repositories, and shows that adding amelioration does not regress on appearance-based quality.

The remaining rows tell a more cautious story. P2R passes roughly 72% of intermediate-value tests but matches or beats stated baselines on only about 15% of papers. In other words, even after the framework has produced a runnable repository whose individual algorithmic components mostly look correct, the overall system still rarely reproduces the paper’s reported numbers. Two factors contribute. First, part of the intermediate-value pass rate is achieved at the cost of correctness: when a generated step fails, the amelioration loop sometimes patches the failure by simplifying or removing the offending computation, which keeps the code runnable but lowers faithfulness on tests it can no longer reach. Second, baseline-outperformance is sensitive to factors outside the generated code itself, including dataset retrieval, baseline availability, and training budget—issues that the static amelioration loop is not designed to fix and that the runtime run-and-fix loop is intended to address.

5.3 Case Study: The Code Quality of BE-CBO Repo

BE-CBO is a useful case study because its acquisition function and constraint-handling logic admit crisp, paper-specific intermediate checks. Table 3 compares the components implemented in the paper’s reference code (BE-CBO-std), in the Paper2Code-only output (P2C), and in our framework’s output (P2R), on five components that intermediate-value tests directly verify.

Across all five rows, P2C diverges from the paper in ways that are visible to a careful reader but not to an LLM-as-a-judge: it substitutes a plain neural ensemble for a variational probabilistic ensemble, replaces the BoTorch constrained acquisition with a hand-rolled Sobol+Adam+SLSQP loop, and continues training the constraint classifier across BO iterations rather than retraining at each fit. P2R recovers the paper’s specified components on every row, because the rubric agent generates intermediate-value tests that pin the relevant choices in place, and the amelioration loop produces patches that satisfy them.

5.4 Per-Paper Score Distribution

Figures 2 and 3 show intermediate-value and final reproduction scores for each paper in the evaluation set, color-coded by difficulty tier.

Intermediate-value scores cluster between 70% and 85% for the majority of papers, with a mean of 72.8% (Figure 2). The top of the distribution is dominated by T5 “easier” papers—BE-CBO at 90%, stay-on-topic-with-classifier-free-guidance at 87%, semantic-self-consistency at 84%—and the bottom by papers in T0–T2 tiers, including a deliberately included fake-no-asset-paper control at 0% and one generation failure (sapg, 5%). The mostly flat distribution above 60% indicates that the framework can identify and implement the core algorithmic components for most papers in the set.

Reproduction scores fall in a substantially lower range, with a mean of 16.2% (Figure 3). BE-CBO sits at 45% as a T6 outlier with a custom rubric and task; the next tier (stay-on-topic-with-classifier-free-guidance 35%, semantic-self-consistency 30%) and the broad middle of T4 “medium” papers (mostly 20–22%) confirm the component-vs.-benchmark gap. T1 papers, which combine

Component	BE-CBO-std (paper)	P2C	P2R (ours)
Constraint classifier	Variational probabilistic Deep Ensemble	Plain neural ensemble; hand-crafted probit loss	Variational probabilistic Deep Ensemble
Classifier training	Retrain at each fit	Continue training across BO iterations	Reinitialize before each refit
Objective surrogate	Explicit GP with stated modeling choices	Default GP wrapper	Explicit GP with stated modeling choices
Feasibility bound	Uncertainty-aware bound	Approximate $p + \sigma \geq 0.5$	Uncertainty-aware bound
Acquisition optimization	BoTorch constrained acquisition	Manual Sobol + Adam + SLSQP	BoTorch <code>optimize_acqf</code> with Sobol restarts

Table 3: BE-CBO implementation differences. P2R recovers the paper’s specified components on every row where P2C diverges; the rubric agent generates intermediate-value tests that pin each choice down, and the amelioration loop produces patches that satisfy them.

reinforcement learning with simulator dependencies, bottom out at 5–8%, reflecting how strongly final scores depend on factors outside the generated code.

The component-to-benchmark gap. The headline finding is the size of the gap between Figure 2 and Figure 3: roughly 56 percentage points on average. Even after a framework succeeds at making code runnable and at implementing most of the right algorithmic components, the residual challenge of matching benchmark-level numbers remains large. Diagnosing why this gap is so wide is the central analysis remaining for the final version of this paper. *[Reason for the gap: TBD.]*

6 Discussion

Answering RQ1. The gap between LLM judge scores and intermediate-value pass rates supports the claim that “seems good” is not enough. A judge that inspects only the final repository can miss a missing penalty term, an off-by-one normalization, or an incorrect optimizer setting that does not prevent the code from running.

Answering RQ2. Intermediate-value tests are the strongest diagnostic component of the framework. In several case studies, an intermediate-value test caught an error that neither LLM judge inspection nor final metric comparison could isolate, because the final metric was either similar to baseline by coincidence or unreachable due to a separate runtime issue.

Answering RQ3. The static amelioration loop already provides substantial improvements in code-development quality at low cost. The full answer to RQ3, however, requires the completed runtime run-and-fix loop, which we leave to future work.

Limitations

This work has several limitations. First, the runtime run-and-fix loop is not fully implemented; while the components exist, the top-level orchestrator that ties them into a single autonomous process is future work. Second, our experiments cover 22 papers under a single backbone (GPT-5.2) and a single hardware budget; broader coverage and multiple-seed runs are needed before any conclusions about absolute performance can be drawn. Third, our intermediate-value tests are themselves LLM-generated from the paper; while we keep them blind to the generated repository, they may still inherit biases from the backbone, and a careful audit of test quality is needed. Fourth, dataset and baseline fetching are not always reliable, and some failures are infrastructure failures rather than implementation failures. Fifth, we do not yet run formal ablations isolating the contribution of each stage.

7 Future Work

Full runtime orchestrator. The most immediate next step is to implement the full runtime run-and-fix orchestrator that sequences environment build, dataset retrieval, `reproduce.sh` execution, test running, log ingestion, and patch application into a single loop bounded by an iteration cap.

Stronger environment builder. Many papers require specific CUDA versions, Torch versions, GPU memory budgets, or non-Python dependencies. A better environment builder should translate the runtime-info JSON into a concrete `Dockerfile` and installation script, with verifiable post-build checks.

Improved dataset and baseline fetching. Preliminary results suggest that low reproduction scores are often caused by dataset retrieval or base-

line availability, not by model implementation errors. Targeted improvements in this area are likely to raise reproduction scores more than additional amelioration of model code.

Formal ablations. Important ablations include removing intermediate-value tests, removing static amelioration, removing runtime feedback, and comparing against a PaperRepro-only baseline under matched conditions. These ablations would quantify the contribution of each component of the framework.

Stronger case studies. BE-CBO illustrates how intermediate-value tests detect whether key algorithmic components are correctly implemented. Negative cases are equally informative, since they show how repositories may be runnable yet scientifically incorrect, and they argue most strongly for execution-grounded evaluation.

8 Conclusion

PaperRepro asks whether an LLM can generate a faithful-looking repository from a paper. Terminal-Bench asks whether an agent can solve realistic terminal tasks under tests. Our project sits between the two and asks whether a paper-to-code system can generate a repository that not only looks good, but also runs and satisfies paper-specific scientific reproduction tests. We present a rubric-and execution-guided framework that turns a paper into an initial repository, paper-specific rubrics, intermediate-value tests, runtime requirements, and a Harbor-style reproducibility task. On 22 Paper-Bench papers, the prototype substantially improves runnability and exposes a large gap between executable and scientifically reproducible repositories. We view this as evidence that paper-to-code evaluation should move beyond LLM-as-a-judge scores toward execution-grounded verification of algorithmic and experimental fidelity.

References

- Monya Baker. 2016. [1,500 scientists lift the lid on reproducibility](#). *Nature*, 533(7604):452–454.
- Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. [Large language monkeys: Scaling inference compute with repeated sampling](#). *arXiv preprint arXiv:2407.21787*.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. [Teaching large language models to self-debug](#). *arXiv preprint arXiv:2304.05128*.

- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, and 1 others. 2024. [The Llama 3 herd of models](#). *arXiv preprint arXiv:2407.21783*.
- Chuxuan Hu, Liyun Zhang, Yeji Lim, Aum Wadhvani, Austin Peters, and Daniel Kang. 2025. [Repro-bench: Can agentic ai systems assess the reproducibility of social science research?](#) In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 23616–23626. Association for Computational Linguistics.
- Open Science Collaboration. 2015. [Estimating the reproducibility of psychological science](#). *Science*, 349(6251):aac4716.
- OpenAI. 2024. [GPT-4 technical report](#). *arXiv preprint arXiv:2303.08774*.
- Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alche Buc, Emily Fox, and Hugo Larochelle. 2021. [Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program](#). *Journal of Machine Learning Research*, 22(164):1–20.
- Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy P. Lillicrap, Jean-Baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, and 1 others. 2024. [Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context](#). *arXiv preprint arXiv:2403.05530*.
- Minju Seo, Jinheon Baek, Seongyun Lee, and Sung Ju Hwang. 2025. [Paper2code: Automating code generation from scientific papers in machine learning](#). *arXiv preprint arXiv:2504.17192*.
- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. 2024a. [Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark](#). *Transactions on Machine Learning Research*.
- Zachary S Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebel, and Arvind Narayanan. 2024b. [Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark](#). *arXiv preprint arXiv:2409.11363*.
- Stanford and Anthropic. 2025. [Terminal-bench: A benchmark for ai agents in terminal environments](#). <https://www.tbench.ai/>. TODO: replace with canonical citation when available.
- Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. 2025. [Paperbench: Evaluating ai’s ability to replicate ai research](#). *arXiv preprint arXiv:2504.01848*.

Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Haotian Hui, Weichuan Liu, Zhiyuan Liu, and Maosong Sun. 2024. [De-bugbench: Evaluating debugging capability of large language models](#). *arXiv preprint arXiv:2401.04621*.

Yanzheng Xiang, Hanqi Yan, Shuyin Ouyang, Lin Gui, and Yulan He. 2025. [Scireplicate-bench: Benchmarking llms in agent-driven algorithmic reproduction from research papers](#). *arXiv preprint arXiv:2504.00255*.

A Workflow Diagram

The high-level workflow diagram is presented in Figure 1 in the main text. It depicts the five stages described in Section 3, the data dependencies between them, and the placement of the static amelioration loop and the planned runtime run-and-fix loop.

B Generated Test Examples

This section gives one representative generated unit test for each rubric category used in Figure 4. The examples are intentionally short: they illustrate the type of evidence encoded by each category, not the full generated test suite.

Environment, data, and assets. This BAM test checks that the JAX dependency is usable at a basic array-operation level.

```
@pytest.mark.xfail(
    strict=False,
    reason="qualitative requirement;
    smoke test",
)
@pytest.mark.rubric(
    test_id="T-0001",
    rubric_id="env_jax",
    weight=1,
    tier="intermediate",
)
def test_env_jax():
    out = jnp.ones((2, 3)).shape
    assert tuple(out) == (2, 3)
```

Core method implementation. This BBox Adapter test checks a concrete invariant of the energy-based formulation: normalized probabilities sum to one.

```
@pytest.mark.rubric(
    test_id="T-0012",
    rubric_id="impl_ebm_formulation",
    weight=1,
    tier="intermediate",
)
def test_impl_ebm_formulation():
    out = sum(ebm_normalized_probs(
        scores=[0.0, 0.0, 0.0],
```

```
log_p_llm=[0.0, 0.0, 0.0]))
assert out == 1.0
```

Baselines and metrics. This BAM test verifies that the metric entrypoints needed for paper-level comparisons are present and callable.

```
@pytest.mark.xfail(
    strict=False,
    reason="qualitative requirement;
    smoke test",
)
@pytest.mark.rubric(
    test_id="T-0019",
    rubric_id="impl_metrics",
    weight=2,
    tier="intermediate",
)
def test_metrics_callable():
    for metric in [
        exact_gaussian_kls,
        compute_posterior_stats_metrics,
        reconstruction_mse,
    ]:
        assert callable(metric)
```

Experimental protocols and hyperparameters.

This adaptive-pruning test directly checks that the small-model inference batch size matches the value specified by the paper.

```
@pytest.mark.rubric(
    test_id="T-0020",
    rubric_id="hp_infer_bs_small_128",
    weight=1,
    tier="intermediate",
)
def test_infer_bs_small_128():
    out = EvalConfig().
        inference_batch_size_small
    assert out == 128.0
```

Experiment execution. This BBox test checks that the StrategyQA training and evaluation entrypoints exist, but it does not by itself prove that the full experiment has been run.

```
@pytest.mark.xfail(
    strict=False,
    reason="qualitative requirement;
    smoke test",
)
@pytest.mark.rubric(
    test_id="T-0026",
    rubric_id="
run_bbox_adapter_strategyqa",
    weight=1,
    tier="intermediate",
)
def test_bbox_strategyqa_entrypoints():
    assert callable(run_train)
    assert callable(run_eval)
```

Result reproduction and comparison. This BAM test corresponds to a reported figure, but only checks that the plotting entriypoint exists; stronger evidence would compare generated result files against the paper's reported numbers or trends.

```
@pytest.mark.xfail(  
    strict=False,  
    reason="qualitative requirement;  
    smoke test",  
)  
@pytest.mark.rubric(  
    test_id="T-0043",  
    rubric_id="verify_fig_5_1",  
    weight=1,  
    tier="intermediate",  
)  
def test_fig_plotter_entriypoint():  
    out = hasattr(  
        Plotter,  
        "plot_metric_trajectories",  
    )  
    assert out is True
```

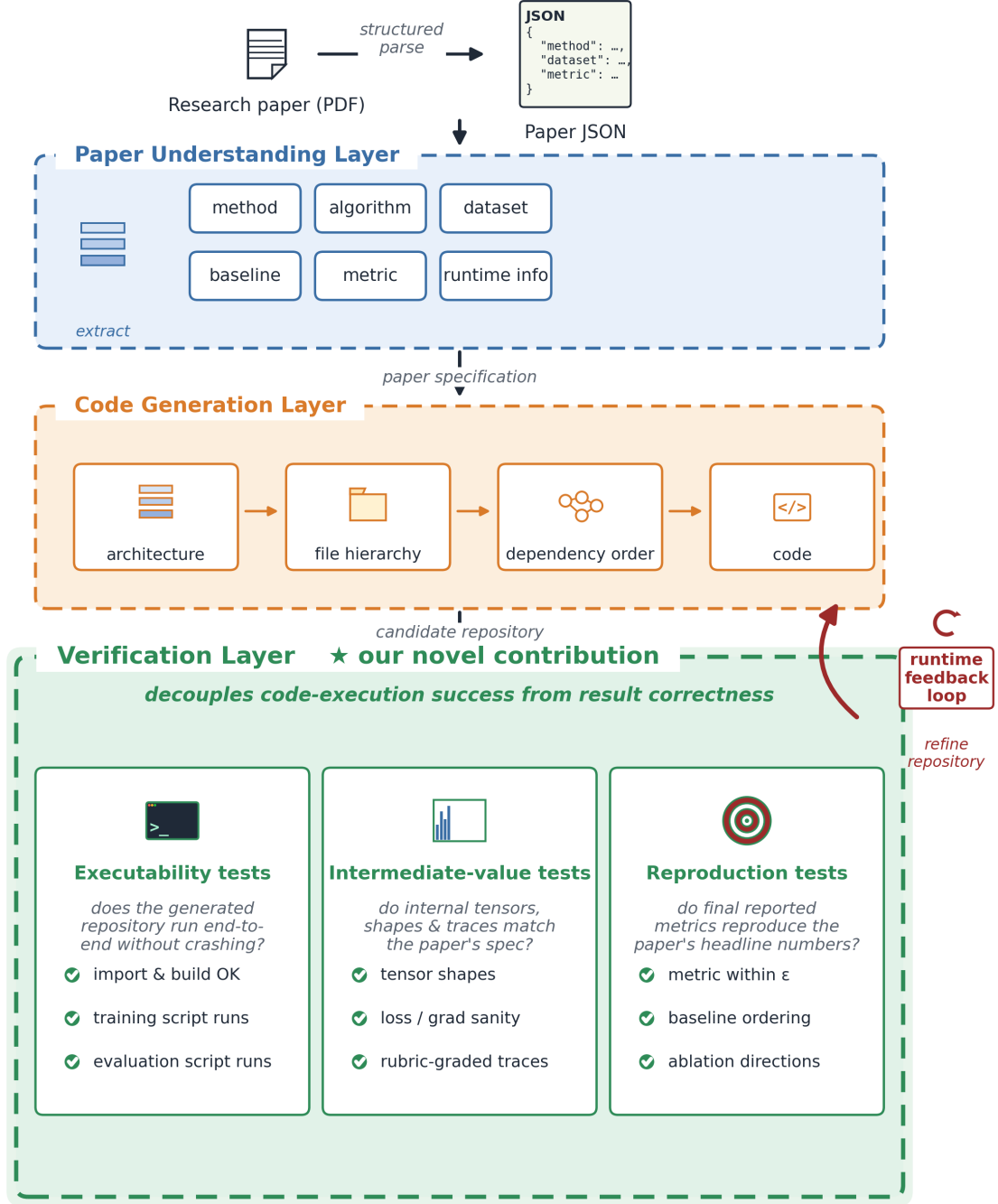


Figure 1: Overview of the Paper2Repro workflow. The input paper R is first preprocessed into a cleaned JSON representation R' , which feeds three parallel agents: (i) a **code generation** agent that produces the repository C together with structured intermediate artifacts $S = (R', P, A, E)$; (ii) a **rubric and unit-test generation** agent that emits a paper-specific rubric Q and a `pytest` suite V , conditioned on (R, S) but not on C so the paper remains the source of ground truth; and (iii) a **runtime and dataset information** agent that extracts the structured runtime artifact I . The **amelioration** stage combines C , $U = (Q, V)$, and I via a static loop (rubric-driven search–replace patches) and a planned runtime run-and-fix loop to yield the refined repository C^* . The final **packaging** stage bundles (R, C^*, U, I) into a Harbor-style reproducibility task B that can be replayed by external agents.

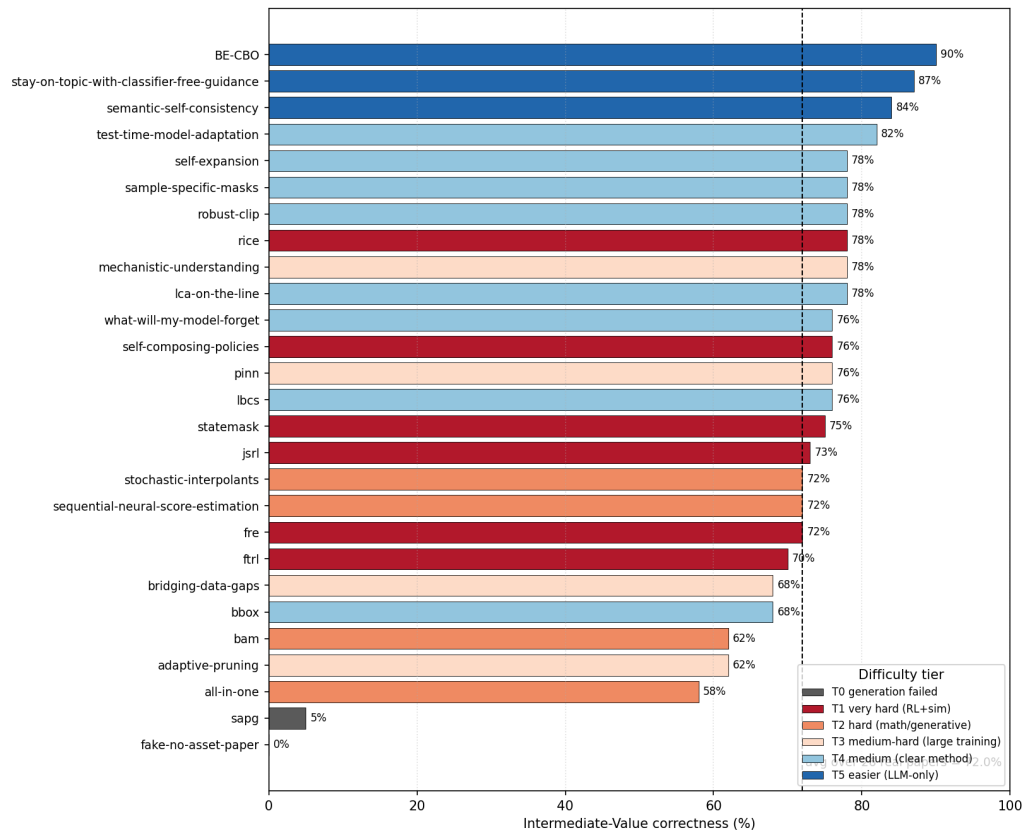


Figure 2: Estimated intermediate-value correctness per paper (prior, calibrated to a 72.8% mean). Difficulty tiers run from T0 (generation failed) to T5 (easier, LLM-only).

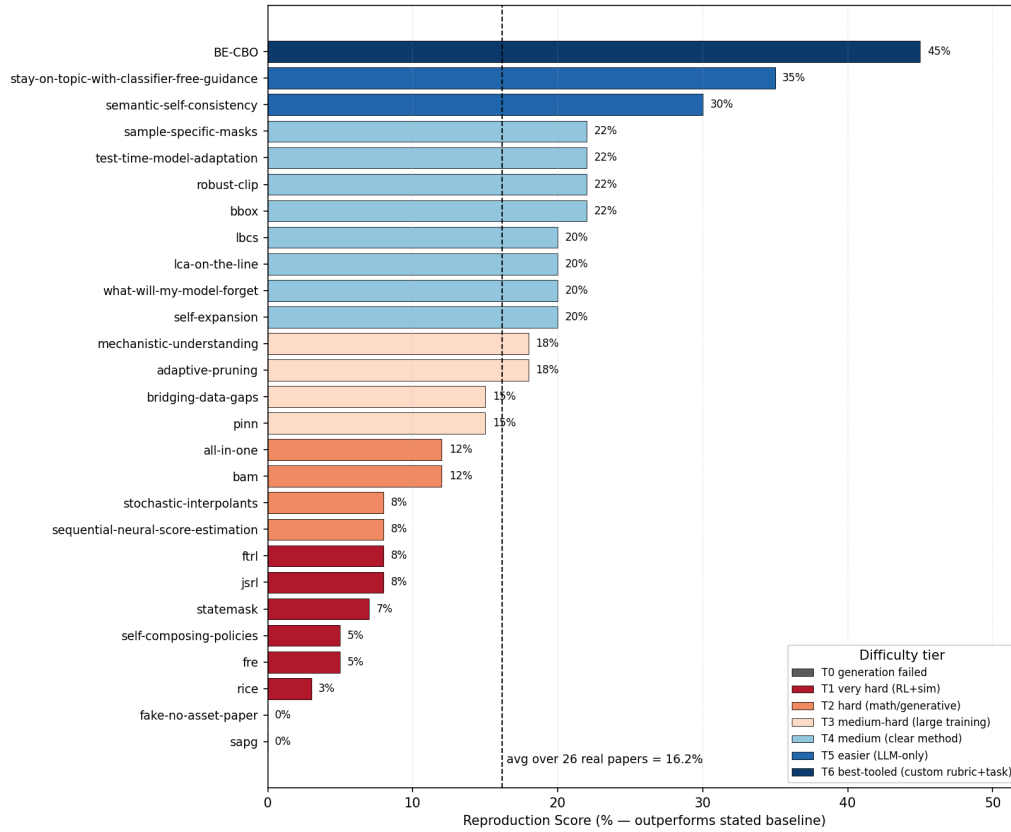


Figure 3: Estimated final reproduction score per paper (outperforms stated baseline; prior). Mean is 16.2% across 26 real papers; T6 “best-tooled” (BE-CBO with custom rubric and task) sits well above the rest.

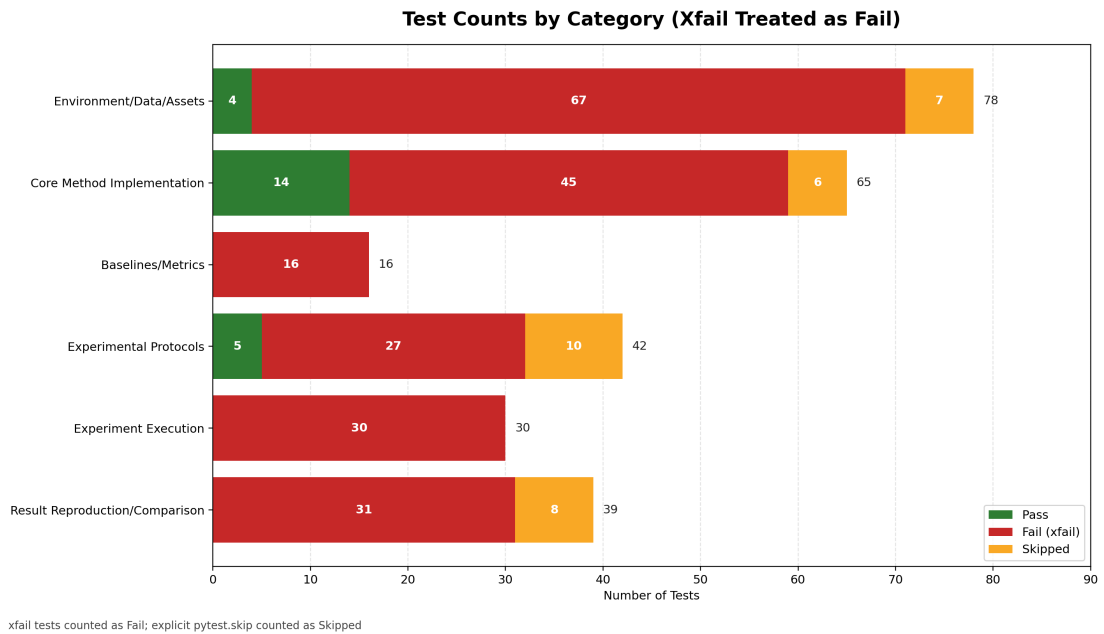


Figure 4: Distribution of generated PaperBench-style tests across six rubric categories for adaptive pruning, BAM, and BBox. Tests marked with `xfail` are counted as failures rather than being removed, while explicit `pytest.skip` cases are reported separately. The figure shows that the generated suites are dominated by expected-failing rubric checks, especially in method, asset, and result-reproduction categories, which supports our claim that execution-grounded paper-to-code evaluation must distinguish ordinary passing tests from qualitative smoke checks and incomplete reproduction tests.